

How Klander works

My study and presentation guide

Ethan Jeknine · October 2026 (updated after phase 11)

Read this to understand every part of Klander well enough to explain it, demo it and answer questions about it.

Companion to: *Klander project report*

Contents

1. Klander in one minute.....	3
2. The big picture.....	4
3. What happens when you add an event.....	5
4. Accounts and signing in.....	6
Email and password.....	6
Continue with Google (OAuth).....	6
5. How privacy works.....	7
6. Friends and invite links.....	8
Adding by username.....	8
Invite links.....	8
Live updates and the activity feed.....	8
7. Repeating events and the clock change.....	9
The clock-change problem.....	9
Skipping one week.....	9
Overnight events.....	9
8. Calendar links (import and subscribe).....	10
The .ics format.....	10
Bringing calendars in (import-ics).....	10
Sending Klander out (feed).....	10
9. Smart add (AI).....	11
10. What changed after phase 5.....	12
11. Themes and birthdays.....	13
Themes.....	13
Birthdays.....	13
12. Plans and finding a time.....	14
The free-time finder.....	14
Invites, polls and "Up for something".....	14
Close friends and surprises.....	14
13. Chat.....	15
14. Memories and My Week.....	16
15. Notifications.....	17
How a notification reaches your lock screen.....	17
16. Shifts, family link and quicker editing.....	18
17. Insights.....	19
18. Helping new people.....	20
19. Hosting, deploying and secrets.....	21
20. Where things are in the code.....	22
21. Costs and limits.....	23
22. Presenting Klander.....	24
3-minute talk (about 3½ with the new features).....	24
Demo checklist.....	24
23. Questions you might be asked.....	25
24. Mini glossary.....	27

1. Klander in one minute

Klander is a free calendar app for friend groups. Everyone has their own calendar, but you can see your friends' weeks, with privacy you control on every event. You can add plans by typing a sentence like "basketball thurs 7pm" or by taking a photo of a poster or timetable, and AI turns it into events. It pulls in your uni timetable and other calendar links, and your family can follow your plans from Apple Calendar. It installs on an iPhone straight from Safari, with no App Store, and costs nothing to run.

Since then it has grown plans with invites, chat, photo memories, lock-screen notifications and private stats (chapters 10–18). If you only remember three things:

1. **Privacy lives in the database.** The app can't leak what the database won't hand over.
2. **It works with the calendars people already use** by reading and publishing standard .ics calendar links.
3. **AI drafts, the human decides.** Smart add never saves anything until you've checked it.

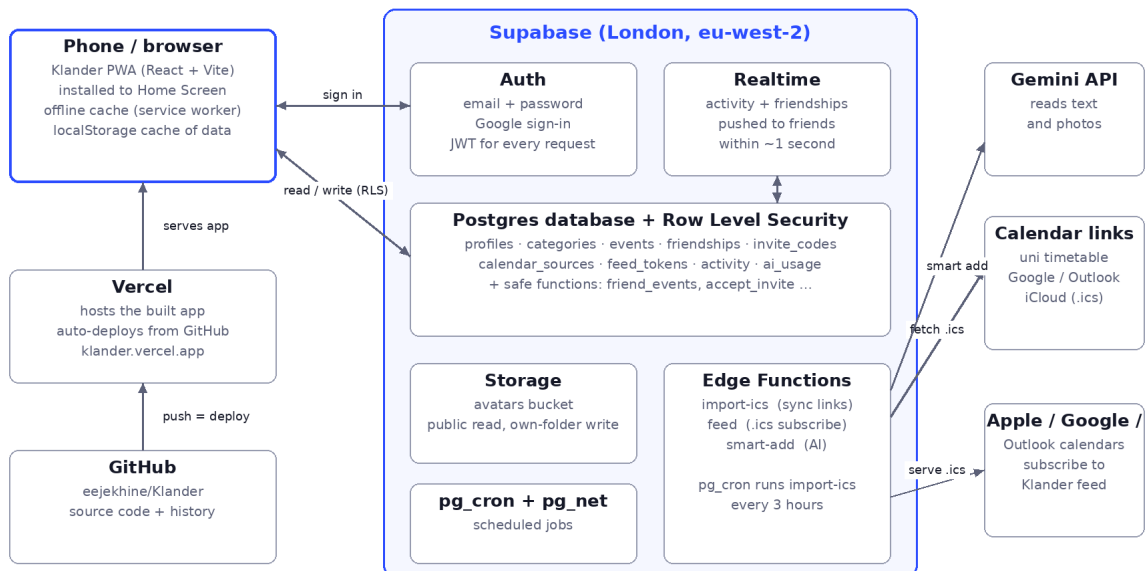
2. The big picture

Think of Klander like a **restaurant**:

Restaurant	Klander	What it does
The menu and the waiter	The app on your phone (React)	Shows you everything and takes your orders
The kitchen's storeroom	The database (Postgres on Supabase)	Where everything is kept
The bouncer on the storeroom door with a guest list	Row Level Security (RLS)	Checks who you are and only hands over what you're allowed to have
Your wristband	Your login token (JWT)	Proves who you are on every request
The chefs in the back	Edge Functions	Do the jobs that need secret ingredients (API keys) the waiter mustn't hold
The delivery driver on a timer	pg_cron	Goes out every 3 hours to fetch your uni timetable
A specialist consultant	Gemini AI	Reads your sentence or photo and suggests events
The building the restaurant is in	Vercel	Hosts the app so anyone can open <code>klander.vercel.app</code>

Klander system architecture

Everything runs on free tiers. The phone talks to Supabase directly; secret work happens in Edge Functions.



How the parts connect

3. What happens when you add an event

Follow "Gym, Tuesday 17:00" from your thumb to your friend's screen:

1. You tap **+**, fill in the editor and press **Save**.
2. The app sends the event to Supabase with your **login token** attached.
3. The database's **RLS rule on events** checks that `owner_id` equals the user in your token. If someone tried to save an event as you, it would be refused here.
4. The event is saved as one row. If it repeats, the repeat rule is saved too (e.g. weekly on Tuesdays), not hundreds of copies.
5. A **database trigger** notices the new event and writes a line in the `activity` table: "Ethan added Gym, Tue 17:00". It applies privacy as it writes: busy events get no title, and private events get no line at all.
6. **Realtime** sees the new activity line and pushes it to any friend who has Klander open. The RLS rule on activity means only your friends receive it.
7. Your friend's app reloads your events by calling `friend_events()`, which hands over only what that friend is allowed to see. Gym appears in your colour within about a second.

4. Accounts and signing in

Email and password

- Supabase Auth stores your email and a **hashed** password. Even the database owner never sees the real password.
- When you sign up, a **trigger** automatically creates your profile and five default categories (Work, Uni, Sport, Social, Personal).
- When you sign in, Supabase gives the app a **JWT (login token)**: a signed note that says "this is user 1234" and expires after an hour. The app refreshes it automatically.

Continue with Google (OAuth)

1. You tap the button. The app sends you to Google with Klander's **Client ID**.
2. You sign in with Google and agree to share your name and email.
3. Google sends you back to Supabase's callback address with a one-time code. That's why the **redirect URI** had to be set in Google Cloud: Google only sends codes to addresses on the list, which is what caused the `redirect_uri_mismatch` error.
4. Supabase swaps the code for your Google details using the **Client Secret**, which only Supabase knows, creates or finds your account, and sends you back to `klander.vercel.app` signed in.

5. How privacy works

Every table has **Row Level Security** switched on. Out of the box, nobody can see anything; each rule opens up one specific thing. For example, the rule on events is:

```
create policy "Users manage own events" on public.events for all to authenticated
  using ((select auth.uid()) = owner_id)          -- you can only see rows you own
  with check ((select auth.uid()) = owner_id);    -- you can only save rows as yourself
```

So how do friends see your events if they can't read the events table? Through one carefully written **database function**, `friend_events()`:

- It only looks at people you have an **accepted** friendship with.
- It **drops** Private events.
- It **rewrites** Busy events to just "Busy" with no location.
- It **never** returns notes.

You set the event to...	Your friend sees...
Friends	Title, time and place
Busy only	A striped block called "Busy"
Private	Nothing

Why this matters (good interview point): if privacy were only hidden in the app, anyone who knew how to call the API directly could read everything. Because the database itself enforces it, even a bug in the app or a technical user can't see more than they're allowed. I tested this by pretending to be different users and checking what each one got back.

6. Friends and invite links

Adding by username

- `send_friend_request("sam_k")` runs in the database. It refuses yourself and usernames that don't exist, and won't create duplicates.
- If Sam had already sent you a request, it just accepts it.
- Only the person who **received** a request can accept it. The rules even stop people editing any column except "status".

Invite links

- Everyone has a random invite code that only they can see, e.g. `klander.vercel.app/?invite=3f9a....`
- When someone opens it, the app saves the code until they've signed up or signed in, then calls `accept_invite(code)`, which makes them friends straight away.
- Because whoever has the link becomes your friend, the app warns you to only share it with people you want to add.

Live updates and the activity feed

- A **trigger** on events writes privacy-safe lines to `activity`.
- **Realtime** pushes new activity lines and friend-request changes to open apps.
- The app waits 0.4 seconds, then refreshes (so 5 quick changes cause 1 refresh).
- It also refreshes every minute and whenever you reopen the app, as a backup.

7. Repeating events and the clock change

A repeating event is stored **once** with a rule, like `FREQ=WEEKLY;BYDAY=MO,TU,WE,TH,FR` ("every weekday"). When you look at a week, the app works out the actual dates.

The clock-change problem

Computers store times in **UTC**. My shift starts at 23:30 in London:

- In October (British Summer Time, UTC+1) that's **22:30 UTC**.
- After the clocks go back on 25 October (GMT, UTC+0) it's **23:30 UTC**.

If the app just added "1 week" in UTC, the shift would show at 22:30 after the clocks changed. The fix is **floating time**: work out the repeats using the local wall-clock time (always 23:30), then convert each one back to the real moment. A test checks this across 25 October.

Skipping one week

"Delete only this one" doesn't delete the series. It adds that date to the event's list of **skipped dates** (`exdates`), so the app jumps over it when working out the repeats.

Overnight events

If you set an end time earlier than the start (23:30 → 08:30), the editor moves the end to the next day automatically. The week view splits it across both days.

8. Calendar links (import and subscribe)

The .ics format

Almost every calendar app can share a calendar as an **.ics file** at a link. `webcal://` is the same thing; it just tells your phone "open this in a calendar app". It's plain text, like this:

```
BEGIN:VEVENT
UID:lec-1
SUMMARY:Intro to Data Science
DTSTART;TZID=Europe/London:20261005T100000
RRULE:FREQ=WEEKLY;BYDAY=MO
END:VEVENT
```

Bringing calendars in (import-ics)

1. You paste a link. Klander saves it in `calendar_sources`.
2. The **import-ics Edge Function** downloads it on the server. Your phone can't do this, because browsers block downloading from other websites.
3. It reads every event, fixes time zones (including Outlook's odd names like "GMT Standard Time"), keeps repeats as rules, handles moved and cancelled lectures, and ignores events older than 4 months.
4. It **upserts** by the event's UID: new ones are added, changed ones updated, and ones that vanished from the feed deleted.
5. **pg_cron** runs this every 3 hours for everyone. **Sync now** runs it straight away.

Sending Klander out (feed)

- Klander gives you a secret link: `.../functions/v1/feed?token=<36 random characters>`.
- Apple, Google or Outlook check that link regularly. The **feed** function returns your Klander-made events as an .ics file.
- It writes times in London time with a time-zone definition, so other apps also get the clock change right.
- Imported events aren't included, because they already live in the calendar they came from.
- Anyone with the link can read it, which is why there's a "make a new link" button that kills the old one.

9. Smart add (AI)

1. You type a sentence and/or choose a photo. The phone **shrinks the photo** (max 1600 px) so it uploads fast.
2. The app calls the **smart-add Edge Function**, sending the current date and time so "thurs" can be worked out.
3. The function checks you're signed in and haven't used your **30 requests** in the last 24 hours.
4. It sends **Gemini** the photo or text plus a list of rules (UK dates, 24-hour times, overnight shifts end the next day, never make things up) and a **JSON schema**: an exact form the answer has to fill in.
5. If one Gemini model is busy, it tries the next one.
6. It checks the answer (real dates, real times) and sends back **drafts**.
7. You see editable cards. Anything the AI wasn't sure about is flagged, and nothing is saved until you tap **Add**.

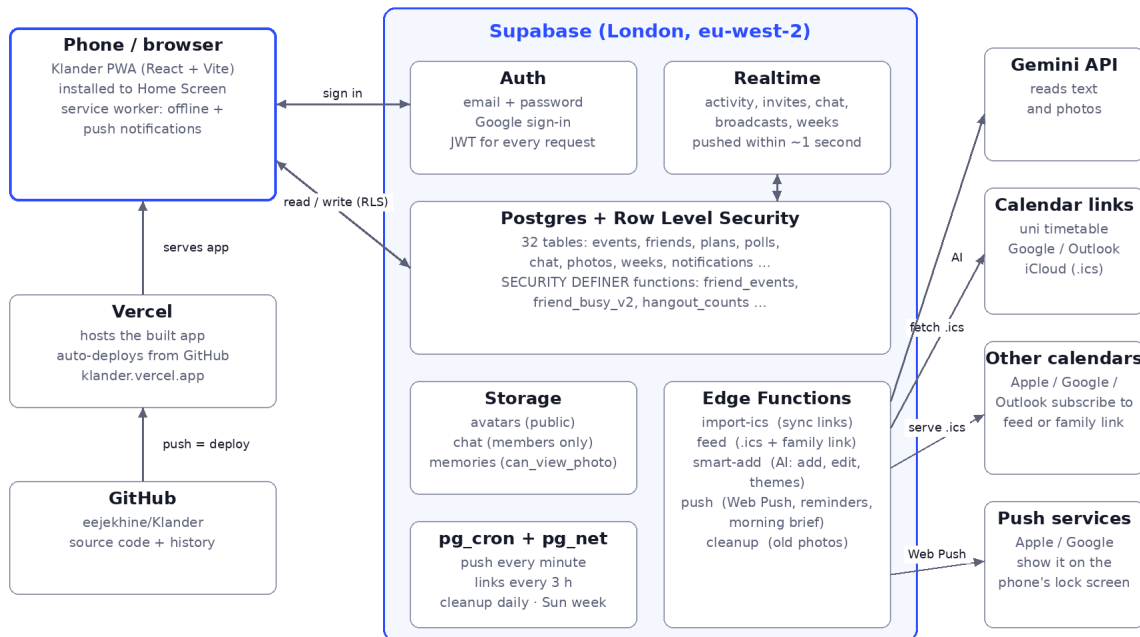
Why the API key is in an Edge Function: if the Gemini key were in the app, anyone could open the browser tools, copy it, and run up usage on my account. On the server it stays secret, and the daily limit stops abuse.

10. What changed after phase 5

Phases 1–5 made Klander a calendar you share with friends. Phases 6–11 made it the place plans actually happen. The same three rules still hold: **privacy in the database, secrets on the server, AI drafts and you decide.**

Klander system architecture (phase 11)

Everything runs on free tiers. The phone talks to Supabase directly; secret work happens in Edge Functions.



How the parts connect now

Phase	In one line
6 Themes & birthdays	Make it yours; never miss a birthday
7 Friends & plans	Find a time, invite people, vote, close friends, surprises
8 Chat	Messages inside the app, with a chat for every plan
9 Memories	Photos on events and a weekly "My Week" story
10 Notifications & sync	Lock-screen alerts, reminders, sleep after shifts, family link
11 Insights	Private stats, streaks and an end-of-term Wrapped
Extras	Swipeable redesign, welcome tour, Help & tips, Add to Home Screen guide

11. Themes and birthdays

Themes

- A theme is just a set of **colour tokens** (`--bg`, `--ink`, `--accent...`). Changing theme swaps the tokens and everything restyles.
- **Contrast guard**: whatever colours someone picks, the app checks the contrast ratio between text and background (the same maths as the accessibility guidelines) and nudges the text colour until it's readable. 200 automated checks cover every theme in light and dark at four times of day.
- **Theme from a photo**: the phone shrinks the photo and groups its pixels into 5 main colours using **k-means** clustering. It never uploads the photo.
- **Theme codes** like `KL1-...` are the theme settings encoded as text. When someone pastes one, it's cleaned first: only known settings and real hex colours are kept, so a code can't smuggle anything in.

Birthdays

- You add your birthday once. Friends get it from `friend_birthdays()`, which returns the year only if you allowed it.
- Birthdays are worked out every year on the phone, not saved as events. A 29 February birthday shows on 28 February in other years.

12. Plans and finding a time

The free-time finder

1. The app asks the database for friends' **busy times only** (`friend_busy_v2()`): start, end and whether it repeats. No titles, places or notes.
2. It expands repeats (including night shifts across the clock change), then merges overlapping blocks for each person.
3. For each day it looks for gaps everyone shares that are long enough, between the hours you chose.
4. It also finds "everyone except one person" gaps, which is often the best you can get.

Interview point: "Find a time" is useful precisely because it doesn't need to know what anyone is doing, only when. That's privacy by design: the data that isn't needed never leaves the database.

Invites, polls and "Up for something"

- **Invites** are rows in `event_invites`. The database only lets you invite friends to your own events, and a guest can only change their own answer.
- **Polls:** you pick 2–5 times; invitees vote yes / maybe / no; you pick the winner and it becomes an event.
- **"Up for something?"** is a short post that expires. Friends reply "I'm in".

Close friends and surprises

Visibility	Close friends see	Other friends see
Friends	Everything (not notes)	Everything (not notes)
Close	Everything (not notes)	"Busy"
Busy only	"Busy"	"Busy"
Private	Nothing	Nothing

A **surprise** adds people to the event's `hidden_from` list. They don't see it at all and can't be invited by mistake.

13. Chat

- Three kinds of chat: **direct** (one per pair of friends), **group**, and **event** (one per plan, for the host and guests).
- Every rule starts with "**are you a member of this chat?**". Non-members can't read or post, and you can only send messages as yourself.
- **Blocking** stops direct messages both ways and hides the person's notifications.
- **Photos** go in a private storage bucket that only members can read. A cleanup job deletes them after 6 months, or straight away if the message is deleted.
- **Make an event** from a message sends its text to Smart add, so "ball at 7 thursday" becomes a draft event.

14. Memories and My Week

- Anyone going to a plan can add photos. They're shrunk on the phone and stored in a private bucket.
- `can_view_photo()` decides who can see each photo: you, people who can see that event, or friends watching a week you posted.
- **My Week** is built on the phone from your week's events and photos. Private and busy-only things are left out by default. Photos you hide are saved **without** their file path, so nobody can load them even with the API.
- **Save image** draws a 1080×1920 collage on a canvas and opens the phone's share menu.

15. Notifications

How a notification reaches your lock screen

1. When you turn notifications on, the phone gives Klander a **push subscription**: an address at Apple's or Google's push service.
2. Something happens (an invite, a message). A **database trigger** writes a row in [notifications](#). The app never writes these rows itself.
3. Every minute, **pg_cron** calls the **push** Edge Function. It finds rows that are due, checks quiet hours and sleep after shifts, and sends each one using its **VAPID** key (which proves it came from Klander).
4. Apple or Google delivers it to the phone, and the **service worker** shows it, even with Klander closed.
 - **Grouping**: friend-event alerts wait 3 minutes, so five quick edits make one notification.
 - **Reminders** come from the event, then its category, then your default. Each gets a unique [dedupe_key](#), so it can never be sent twice.
 - **iPhone**: Apple only allows this for web apps added to the Home Screen, which is why there's an install guide.

Best bug story: reminders passed every mock test but were never created on the real server. The "only once" rule used a partial index that Postgres can't use for upserts, so every insert silently did nothing. A live test on my own phone found it, and a unique [dedupe_key](#) fixed it.

16. Shifts, family link and quicker editing

- **Shifts:** mark a category as shifts and every shift gets a Sleep block afterwards (7 hours by default). It's calculated from each occurrence, so it's still right after the clocks change.
- Friends' sleep counts as busy in the free-time finder, and notifications can wait until you're awake.
- **Family link:** a second secret calendar link. Friends events show in full, Close and Busy-only show as "Busy", Private never appears, and notes are never included.
- **Morning brief:** an optional daily notification with your day, birthdays and who's free tonight.
- **Change by typing:** "move gym to 7" goes to Smart add in edit mode, which returns a proposed change that you approve. Moving a date on a repeating event skips that one occurrence and adds a moved copy.
- **Clash warnings** and **Undo** after deleting (the deleted event and its guest list are kept for 6 seconds so they can be put back).
- **Why no two-way Apple sync?** It would need an Apple app-specific password, which can read every iCloud calendar. I decided that was too much risk; the family link covers the real need.

17. Insights

- Everything on the Insights page is worked out **on your phone** from your own events. Only you see it.
- **Streaks:** weeks in a row with at least one event in a category. If this week hasn't had one yet, it counts from last week, so a streak doesn't "break" on Monday morning. Shifts don't count.
- **Who you made plans with:** `hangout_counts()` counts plans you and a friend both went to. It only ever returns counts involving you, skips blocked people and returns nothing to strangers.
- **Wrapped:** at the end of each term, a story of your top category, your number one friend, a favourite photo, your longest streak and your busiest week.

18. Helping new people

- **Welcome tour:** six slides the first time you open the app.
- **Help & tips:** searchable, and each tip has a "Show me" button that opens the right screen. It lives in [src/lib/help.js](#), which gets updated with every feature.
- **Add to Home Screen guide:** detects iPhone, Android or computer and shows drawn steps. On Android it can offer a one-tap Install button (the browser's [beforeinstallprompt](#) event). It reminds iPhone users to sign in again, because the Home Screen app keeps its own sign-in.

19. Hosting, deploying and secrets

Thing	Where it lives	How it changes
App code	GitHub: eejeknine/Klander	git commit + git push
Live app	Vercel (klander.vercel.app)	Vercel rebuilds automatically on every push (~1 min)
Database structure	Supabase	Named migrations; full copy in supabase/schema.sql
Edge Functions	Supabase	Deployed separately; source in supabase/functions/
Public keys	Vercel environment variables	Supabase URL + publishable key (safe to be public)
Secret keys	Supabase Edge Function secrets	GEMINI_API_KEY; the service role key never leaves Supabase
Push keys (VAPID)	private.app_secrets in the database	Made by the push function on first run; the private half never leaves the server
Scheduled jobs	pg_cron in Supabase	push every minute, links every 3 h, cleanup 03:17 daily, week-ready Sundays
Google sign-in	Google Cloud project "Klander" + Supabase provider settings	Client ID/secret, redirect URI, JavaScript origins

Why the app sometimes looks old after a push: it's a PWA, so it saves itself for offline use. After a deploy, the first open can show the saved version. It updates in the background, and the next open has the new version.

20. Where things are in the code

File / folder	What's in it
src/App.jsx	Decides which screen to show: loading, sign in, reset password or the app
src/Shell.jsx	Loads your data; shows first-time setup if you have no username
src/CalendarApp.jsx	The main screen: header, views, people strip, + and Smart add buttons, all sheets
src/views/	TimeGrid (day/week), MonthView, AgendaView (list)
src/components/	Sign in, onboarding, event editor, settings, friends, calendars, smart add, event viewers
src/lib/dates.js	Repeat expansion, overlap layout, date formatting, repeat presets
src/lib/store.js	Loading/saving your profile, categories and events + offline cache
src/lib/friends.js	Friends, requests, friend events, activity, live updates, invite links
src/lib/sync.js	Linked calendars and your subscribe links
src/lib/smart.js	Photo shrinking, calling smart add, turning drafts into events, edit by sentence
src/lib/themes.js · birthdays.js	Themes, contrast guard, theme codes, photo themes; birthdays and UK dates
src/lib/freetime.js · plans.js · social.js	Free-time finder; invites, broadcasts; close friends, groups, polls
src/lib/chat.js	Chats, messages, reactions, photo shrinking for chat
src/lib/memories.js · week.js	Event photos, My Week builder, friends' weeks, One year ago
src/lib/notify.js · sleep.js	Notifications and settings; sleep blocks after shifts
src/lib/insights.js	Stats, streaks, hangouts, Wrapped
src/lib/help.js · install.js	Help topics and tour; Add to Home Screen detection
public/push-sw.js	Shows push notifications and opens the right screen when tapped
src/styles/app.css	All styling, light and dark colour tokens
supabase/functions/	import-ics, feed, smart-add, push, cleanup, and the shared .ics parser
supabase/schema.sql	The whole database structure and security rules
vite.config.js	Build settings and the PWA manifest (name, icons, colours)

21. Costs and limits

Service	Cost now	Limit to know
Supabase	£0	Free project pauses after a week of no use; 500 MB database; 1 GB storage shared by chat and memory photos (chat photos deleted after 6 months)
Vercel	£0	Hobby plan is for non-commercial use
Gemini	£0 (free tier)	Free tier may use data for training; switch to paid (~35p/month) before strangers use it
Smart add	—	30 per person per day (my own limit)
Calendar refresh	—	Every 3 hours, or Sync now
Push notifications	£0	Web Push is free; iPhone needs the Home Screen app (iOS 16.4+)

Carbon estimate: roughly 0.2–1.2 kg CO₂e to build and 1.5–5 kg CO₂e a year to run for my friend group (about 10–30 km of driving). These are estimates.

22. Presenting Klander

3-minute talk (about 3½ with the new features)

Time	Say	Show
0:00	"My friends and I could never find a time to meet. Our plans live in group chats and five different calendars. So I built Klander."	Home screen icon, open the app
0:20	"It's a normal calendar first: day, week, month, list. My night shift repeats every weekday and stays at 23:30 even when the clocks change."	Week view with night shifts
0:50	"The point is friends. Each friend has a colour. I control privacy on every event: friends see details, just busy, or nothing. That's enforced by the database, not just hidden in the app."	People strip, a busy block, the editor's privacy options
1:30	"Adding plans is quick. I type 'basketball thurs 7pm at the edge', or take a photo of a poster, and AI drafts the event. I check it, then save."	Smart add live
2:10	"Then it helps us actually meet: Find a time uses only when people are busy, never what they're doing. I invite people, they say Going, and the plan has its own chat."	Plans → Find a time → invite
2:35	"It works with what people already use and fits my life: my uni timetable syncs in, my mum follows a family link in Apple Calendar, and after a night shift it knows I'm asleep."	Sleep blocks, family link
2:55	"It's a PWA on React and Supabase, costs £0 a month, sends lock-screen notifications, and every privacy rule has a test. Next: a privacy page, data export and a public launch."	Insights / Wrapped, then roadmap

Demo checklist

- Have two accounts signed in (phone + laptop private window) to show live updates.
- Pre-load a week with lectures, shifts and a couple of friend events.
- Have a poster or screenshot ready for smart add, plus a backup sentence in case the signal is bad.
- Open the app once beforehand so the latest version is loaded.
- Make sure notifications are on and send yourself a test from the bell, so you can show one arriving.
- Have a plan with a guest list and a chat, a posted My Week, and enough events for Insights to look full.

23. Questions you might be asked

Question	A good answer
Did you write the code yourself?	I built it with Claude, an AI assistant. It walked me through each stage and wrote most of the code, especially the hard parts. I came up with the idea and features, made the design and privacy decisions, set up all the services (Supabase, Google sign-in, Gemini, GitHub, Vercel), deployed it, and tested it. I can explain how every part works.
Why a web app and not an iPhone app?	A native iPhone app needs a £99/year developer account, and free builds expire every 7 days. A PWA installs from Safari for free, works on Android and laptops too, and is one codebase.
How do you stop people seeing private events?	Row Level Security in the database. People can only read their own events table rows. Friends get events through one function that drops private events and turns busy ones into "Busy" before anything leaves the database. I tested it by pretending to be different users.
What's Row Level Security?	Rules attached to each database table that decide, row by row, what the current signed-in user can read or change.
Why Supabase over Firebase?	It's real SQL (Postgres), which suits relational data like friendships and is useful for my degree. It also has RLS, auth, realtime, storage and functions in one free project.
How do live updates work?	A trigger writes an activity line whenever an event changes. Supabase Realtime pushes it to friends who have the app open, and their app refreshes.
What was the hardest bug?	Time. Repeating events had to stay at local time across the clock change, and calendar files from Outlook use Windows time-zone names. I used floating time for repeats and a fallback to London time for unknown zones, both covered by tests.
How do you keep the AI from making mistakes?	Strict rules and a JSON schema; checks on every date and time; low-confidence events are flagged; and nothing saves until the user confirms.
Isn't the API key exposed?	No. It's a server-side secret in an Edge Function. The app only holds the Supabase publishable key, which is safe because RLS protects the data.
How did you test it?	Database security tests as fake users for every privacy rule (34 more for phases 6–11, run against the live database), 269 automated logic checks, live tests of the deployed functions, phone-size UI tests with screenshots, and real use on my own phone. All passing.
What does it cost?	£0 a month on free tiers. The only planned cost is about 35p a month for paid AI before opening it to the public.
What would you do differently?	Set up a separate test database from day one for full end-to-end tests, and design offline editing earlier.
What's next?	Phase 12: a privacy page, download my data and delete my account, a landing page and demo mode, then opening it to more people.
How do notifications work without an app store app?	Web Push. The phone gives Klander a push address; a database trigger queues the notification; a function on a 1-minute timer sends it with Klander's VAPID key; Apple or Google delivers it. On iPhone it needs the Home Screen app.

Question	A good answer
How does Find a time protect privacy?	It only gets busy start and end times from the database, never titles or places, so it can compare everyone without anyone sharing details.
What was the hardest bug after phase 5?	Reminders that silently never got created on the real server. The "only once" rule used a partial index Postgres can't use for upserts. I found it by testing on my own phone and fixed it with a unique key.
Why didn't you build two-way Apple sync?	It needs an Apple app-specific password that can read every iCloud calendar. I decided the risk wasn't worth it and built a read-only family link instead.
Who can see my chats and photos?	Only chat members, enforced in the database and in private storage buckets. Photos you hide from My Week are saved without their file path, so they can't be loaded.
Is it GDPR-safe?	Data is in the EU (London), access is locked down by RLS, and before launch I'm adding a privacy page, data export and account deletion.

24. Mini glossary

Word	Plain meaning
Front end / back end	What you see (the app) / what stores and protects the data (Supabase)
API	The way the app asks the server for things
JWT	Your signed login token
RLS	Per-row security rules in the database
Trigger	Code the database runs automatically when something changes
Edge Function	Small server program for anything that needs secrets
Cron	A timer that runs a job on a schedule
PWA	A website you can install like an app
.ics	The standard calendar file format
Upsert	Insert if new, update if it already exists
Migration	A saved, named change to the database structure
Web Push	How a website sends lock-screen notifications
Service worker	A script the phone keeps for the app: offline and notifications
Contrast ratio	How readable text is against its background (4.5 or more is good)
k-means	Grouping pixels into a few main colours
Dedupe key	A unique label so the same thing can't be created twice